XASM_LGP30 Cross Assembler Manual

Author: Neil Higgins
Applicable to: XASM_LGP30 Version 1.0

Contents

Contents 1
Disclaimer 3
Introduction 3
Why? 3
Support 3
Installation 3
Invocation 4
Terminology 4
Statements 4
Number Formats 5
  Explicit number formats 5
  Implicit number format 5
  Address literals 5
  Types 6
LGP-30 Instructions 6
Assembler Directives 7
Escape Sequences for Friden Flexowriter Characters 8
Case Sensitivity 8
Output formats 8
  Listing 8
  Binary 8
Expressions 9
  Operators 9
  Built-in Functions 9
  Parentheses 9
Notes 10
References 11
Appendix 1 – Instructions in Functional Order 12
Appendix 2 – Friden Flexowriter Character Set 14
Appendix 3 –Test Programs 15

Appendix 4 Demonstration Programs 16
Echo Console Input in 6-bit Mode 16
Source Code 16
Assembly 16
Execution 16
Hello World Program (print strings) 17
Source Code 17
Assembly 22
Execution 22
Evaluate a Polynomial Expression 23
Source Code 23
Assembly 24
Execution 24

Disclaimer

This manual and the XASM_LGP30 cross assembler which it describes are free (as in beer) and free (as in thought). Do with them as you will, provided that you accept all of the consequences.

Introduction

XASM_LGP30 is a cross assembler for the LGP-30 computer. It is written in Python.

It is a two-pass assembler. In Pass 1, all elements are validated structurally, and the symbol table is built. In Pass 2, all elements are re-validated structurally, and program statements are assembled.

The focus of the product is utility, not authenticity. As such, historical LGP-30 conventions are comprehensively abandoned in favour of more modern conventions.

Why?

Why write a cross assembler for an ancient machine with few redeeming characteristics? Mostly because the original utilities look and feel arcane, and can only be used in conjunction with a large repertoire of "memory items". The XASM_LGP30 project was developed in tandem with another project, SIM_LGP30 – an LGP-30 simulator, which can also be found on GitHub.

Support

The program has been written to meet the author's personal requirements. It has been shared for what it is worth, but major changes are not envisaged. If you wish, report installation problems, bugs and feature requests using the relevant GitHub features. This is the only way you will get a response, which in any case is not guaranteed.

Installation

The author has minimal packaging and installation skills. The following should work:

1. Clone the GitHub repository.

git clone https://github.com/etherbrat/xasm_lgp30

2. This will create a structure which looks like:

Structure	Actual
project_dir	xasm_lgp30
dist_dir (distribution)	dist
docs_dir (documentation)	docs
src_dir	src
modules_dir (python files)	xasm_lgp30
tests_dir (tests)	tests
venv_dir (interpreter, libraries)	venv
etc.	etc.

3. The .exe file in the dist directory should run under Windows.

To build from source:

4. In the project directory, type:

python -m venv venv (create a virtual environment)
.\venv\scripts\activate (activate the virtual environment)
pip install -r requirements.txt (install the dependencies)
pip install -e . (the dot is a literal - current directory)

5. In the project directory, type:

pip install pyinstaller
pyinstaller --onefile .\src\xasm_lgp30\assembler.py (this will generate a .exe file in the dist directory)

The generated .exe file should run under Windows. If you want, you can copy the .exe file to a Windows Program Files subdirectory and create shortcut to a command window which has the .exe file on its path.

Invocation

In the command window, type:

assembler <path_to_source_file> [-o < path_to_hex_file] [-l path_to_listing_file]

If the -o option is not provided , no hex file will be produced. If the -l option is not provided, no listing file will be produced.

Terminology

The term “sexadecimal” is used to refer to the LGP-30’s unique flavour of hexadecimal (0 – 9, f, g, j, k, q, w).

Statements

Valid statements must be structured as follows:

<label> <code> <w> <args> <comment> <EoL>

where:

- <label> means text followed by a colon (optional)
- <code> means an LGP-30 opcode or a directive (optional)
- <w> means whitespace
- <args> means zero or more whitespace-separated arguments (optional)
- <comment> means a semicolon followed by text (optional)
- <EoL> means end of line

Arguments may be expressions where this is sensible (see below); but note that the statement analyser splits arguments on whitespace, hence expressions may **not** contain embedded whitespace.

Blank lines are permitted.

As with most two-pass assemblers, expressions may not contain forward references if they are evaluated in Pass 1.

Number Formats

Explicit number formats

- binary: “0b” [<sign>] <numbers>, numbers in 0 – 1
- octal: “0o” [<sign>] <numbers>, numbers in 0 – 7
- sexadecimal: “0s” [<sign>] <numbers>, numbers in 0 – w
- hexadecimal: “0x” [<sign>] <numbers>, numbers in 0 – f

Base 16 representations are complicated by the inclusion of the spacer bit as a zero-valued LSB. Fortunate side effects of this include that (a) a word can be represented with an exact number of digits, and (b) the opcode aligns perfectly with a digit.

Implicit number format

Unprefixed numbers are parsed and evaluated as decimals. Floating point numbers must include leading/trailing zeros around the decimal point, e.g.

- 23.0, not 23.
- 0.23, not .23

Scientific notation (e.g. 1.23e4) is **not** accepted.

Address literals

In the LGP-30 documentation, memory addresses are typically expressed in dual-(decimal modulo 64) format, i.e. two decimal digits in 0 – 63 for the track number, concatenated with two decimal digits in 00 – 63 for the sector number. With a view to supporting simple, consistent arithmetic for all numeric expressions, XASM_LGP30 does **not** support this format – address values occupy the continuous range of 0 – 4095.

A further difference from LGP-30 tradition is that the p (print) instruction accepts only a track operand (the character code), not a track/sector operand. In any case, it is expected that users will prefer to use the .pchar assembler directive, which takes a character or escape sequence literal as the operand, and assembles the corresponding p instruction.

Similarly, the z (stop) instruction accepts only a track operand (the logical OR of the switch masks), not a track/sector operand.

Types

Type	Pass 1*	Pass 2
symbol	[_](A-Za-z)[A-Za-z0-9_]*	[_](A-Za-z)[A-Za-z0-9_]*
label	symbol	symbol
number	numeric literal	numeric literal
expression	number or mathematical formula	number or evaluated mathematical formula
integer_expr	expression	integer expression
address_expr	expression	integer_expr in 0 – 4095
word_expr	expression	integer_expr in -2^{31} – 2^{31}-1
q_expr	expression	integer_expr in 1 – 30
shift_expr	expression	integer_expr in 1 – 30
char_code	expression	integer_expr in 0 – 63
character	character in Flexowriter output character set	character in Flexowriter output character set
switch_mask	expression	integer_expr in sw32 | sw16 | sw8 | sw4

* except as noted for the .blkw and .equ directives

LGP-30 Instructions

See also Appendix 1.

Binary	Decimal	Instruction	Mnemonic	Arg Field
0000	0	Stop	z <switch_mask>	Track
0001	1	Bring	b <addr_expr>	Address
0010	2	Store address	y <addr_expr>	Address
0011	3	Return address	r <addr_expr>	Address
0100	4	Input	i 0	Track
0101	5	Divide	d <addr_expr>	Address
0110	6	Multiply lower	n <addr_expr>	Address
0111	7	Multiply upper	m <addr_expr>	Address
1000	8	Print	p <char_code>	Track
1001	9	Extract	e <addr_expr>	Address
1010	10	Unconditional transfer	u <addr_expr>	Address
1011	11	Test	t <addr_expr>	Address
1100	12	Store and hold	h <addr_expr>	Address
1101	13	Store and clear	c <addr_expr>	Address
1110	14	Add	a <addr_expr>	Address
1111	15	Subtract	s <addr_expr>	Address

Assembler Directives

Directive	Description
.atq <expression> <q_expr>	Assemble a word containing the value of <expression> at a q value equal to the value of <q_expr>. <expression> can have an integer or floating point value
.blkw <integer_expr>	Assemble a number, equal to the value of <integer_expr>, of zero-valued words. **Caution**: Although an expression is permitted, its value must not change between Pass 1 and Pass 2
.end <addr_expr>	Stop assembling and set the start address to the value of <addr_expr>
.equ <symbol> <expression>	Create a symbol with the value of <expression>, **in Pass 1.** <expression> can have an integer or floating point value. Multiply defined symbols are not permitted
.locn <addr_expr>	Set the location pointer to the value of the <addr_expr>. **Caution**: Although an expression is permitted, its value must not change between the first and second passes. Moving the pointer backwards is an invitation to disaster
.lshn <shift_expr>	Assemble a word for use by the n multiply instruction to left shift A/ac by the value of the <shift_expr> places
.pchar <character>	Assemble a p instruction to print <character>. The list of escape codes for printing and non-printing characters appears below
.rshm <shift_expr>	Assemble a word for use by the m multiply instruction to right shift A/ac by the value of the <shift_expr> places
.word <integer_expr>	Assemble a word as follows: Evaluate <integer_expr>. Chop the value to **32 bits**, and mask the LSB to zero. Recommended for bitwise operations only. The .atq directive should be used for numerical constants

Escape Sequences for Friden Flexowriter Characters

For use with the .pchar directive.

Seq	Character
#1	colon
#2	semicolon
#~	space
#b	backspace
#c	colour shift
#d	Δ (delta)
#l	lower case
#n	carriage return
#p	∏ (pi)
#r	start reader
#s	Σ (sigma)
#t	tab
#u	upper case
#x	delete

Case Sensitivity

The assembler is case-insensitive except for symbols. The presentation of characters on the printer is entirely dependent on case and colour, which must be explicitly set. The colour shift character is not idempotent; hence the initial colour must be manually set.

Output formats

Listing

Line number, Input line, Opcode, Operand address, Assembled binary, Errors

Binary

Drum images are exported to files in Intel Hex format. Intel Hex format is a time honoured, well documented, robust and easy-to-use format for application binaries.

The format is 32 bits (8 **hex**adecimal bytes), big-endian, **LSB** set to zero. Addresses are **linear**, i.e. address 0xff is literally address 255 (track 3, sector 63).

Expressions

Operators

Operator	Description	Precedence	Associativity
**	Exponentiation	4	Right
*	Multiplication	3	Left
/	Division	3	Left
//	Integer division*	3	Left
+	Addition	2	Left
-	Subtraction	2	Left
unary -	Negation	100	Right

* Calculates float then rounds DOWN, e.g. 7/8 -> 0, -7/8 -> -1 (Python convention)

Built-in Functions

Function*	Description
and(x, y)	Bitwise AND of x and y
or(x, y)	Bitwise OR of x and y
xor(x, y)	Bitwise XOR of x and y
not(x)	Bitwise NOT of x
shl(x, n)	Shift x left n binary positions
shr(x, n)	Shift x right n binary positions
max(...)	Maximum (variable number of arguments)

* Functions can be variadic (e.g. max)

Parentheses

- (and) enforce arithmetic precedence, and delineate function argument lists
- Nested expressions are fully supported

Notes

1. The author has not found any examples of side-by-side source code and binaries. This adds a degree of speculation to the testing process.

References

The following references were all “alive” at the time of writing.

1. LGP-30 Programming Manual – https://www.bitsavers.org/pdf/royalPrecision/LGP-30/LGP-30_Programming_Manual_Apr57.pdf
2. LGP-30 Programming Class Notes – http://computermuseum.informatik.uni-stuttgart.de/ftp/lgp30/docs/LGP30_ProgrammingClassNotes.pdf
3. LGP-30 Subroutine Manual – https://www.bitsavers.org/pdf/royalPrecision/LGP-30/LGP-30_Subroutine_Manual_Oct60.pdf
4. Intel Hex Format - https://en.wikipedia.org/wiki/Intel_HEX

Appendix 1 – Instructions in Functional Order

The following table is a close replica of the original documentation, with a few changes made for clarity.

Mnemonic	Binary	Example	Description
b	0001	b 2000	Bring: Replace the word in the A register with the word in memory location 2000. The contents of memory location 2000 are unaffected.
h	1100	h 2000	Store and hold: Replace the word in memory location 2000 with word in the A register. The contents of the A register are unaffected.
c	1101	c 2000	Store and clear: Replace the word in memory location 2000 with the word in the A register, then zero the A register.
a	1110	a 2000	Add: Add the word in memory location 2000 to the A register and place the result in the A register. The contents of memory location 2000 are unaffected.
s	1111	s 2000	Subtract: Subtract the word in memory location 2000 from the A register and place the result in the A register. The contents of memory location 2000 are unaffected.
m	0111	m 2000	Multiply upper: Multiply the number in the A register by the number in memory location 2000 and place the sign and the most significant thirty bits of the product in the A register. The contents of memory location 2000 are unaffected.
n	0110	n 2000	Multiply lower: Multiply the number in the A register by the number in memory location 2000 and place the least significant thirty-one magnitude bits of the product in the sign bit and thirty magnitude bits of the A register. The contents of memory location 2000 are unaffected.
d	0101	d 2000	Divide: Divide the number in the A register by the number in memory location 2000 and place the quotient, rounded to thirty bits, in the A register. The contents of memory location 2000 are unaffected.
e	1001	e 2000	Extract: (Logical AND) Place zeroes in the word in the A register wherever there are zeroes in location 2000 but otherwise leave the word in the A register unchanged. The contents of memory location 2000 are unaffected.
y	0010	y 2000	Store address: Replace the address portion of the word in memory location 2000 with the address portion of the word in the A register. The contents of the A register are unaffected. The LGP-30 has no index register. This instruction can be used to self-modify code to index an array.
r	0011	r 2000	Return address: Add one to **the (post-fetch) value of** the C register (do **not** update the C register), then replace the address portion of the word in location 2000 with **this value**. R is followed immediately by an unconditional transfer instruction to the beginning of the subroutine. This instruction is used to patch an unconditional transfer instruction at the end of a subroutine, redirecting execution to the instruction following the invoking transfer. R can also be used to patch a Test instruction in order to create a conditional return.
u	1010	u 2000	Unconditional transfer: Replace the word in the C register with the address portion of this instruction.

t	1011	t 2000	Test: (Conditional transfer) If the sign bit of the A register is a 1, the test instruction has the effect of an unconditional transfer. If the sign bit of the A register is a 0, the next instruction in the normal sequence is executed. **But note**: When the transfer control switch is on, if a test **instruction** has the sign bit (normally unused in an instruction word) set, a transfer will take place unconditionally, i.e. regardless of the accumulator's value.
z	0000	z 0000	Stop: Stop computation. If computation is to stop, the address portion of the stop instruction is usually of no significance. However, the stop instruction has a special characteristic: Four break point switches (called Break Point 32, Break Point 16, Break Point 8 and Break Point 4) are supported by the z instruction. There is a 1:1 correspondence between these switches and the four most significant bits of the track number. Computation does not stop if a break point switch is latched down and the corresponding bit is set in the track number.
p	1000	p 2000	Print: Execute the typewriter keyboard function indicated by the 6 track bits. The print order has no effect on the contents of any memory location, the A register, or the C register. For example, p 2000 has 010100 in the track bits which is the code for a back space on the typewriter. The execution of p 2000 results in the typewriter back spacing.
i	0100	i 0000	Input: The address portion of this instruction is always 0000. It is always preceded by the instruction p 0000. After a p 0000 instruction starts the tape reader, an i 0000 instruction transfers into the last 4 bit positions of the A register the first 4 bits of the typewriter code for the first character read on the tape. When the second character is read, the bits representing the first character are shifted left into the next to last four bit positions of the A register and the first four bits of the typewriter code of the second character on tape are placed in the last four bit positions of the A register. This process continues up to eight times to fill the entire A register until a stop code (100000) appears on the tape. The stop code stops the tape reader and sends a start signal to the computer so that the instruction following i 0000 in memory is executed. Often this next instruction is a h or c instruction so that the characters read into the A register can be stored in a memory location.

Appendix 2 – Friden Flexowriter Character Set

Code Order

Alphanumeric Keys		Output Code		Input Code
Upper Case	**Lower Case**	**Decimal (track)**	**6-bit (track)**	**4-bit (6-bit) binary**
Start reader†		0	000000	
Z	z	1	000001	0000 (01)
)	0	2	000010	0000 (10)
Space		3	000011	0000 (11)
Lower case†		4	000100	
B	b	5	000101	0001 (01)
L	1	6	000110	0001 (10)
_	-	7	000111	0001 (11)
Upper case†		8	001000	
Y	y	9	001001	0010 (01)
*	2	10	001010	0010 (10)
=	+	11	001011	0010 (11)
Colour shift†		12	001100	
R	r	13	001101	0011 (01)
"	3	14	001110	0011 (10)
:	;	15	001111	0011 (11)
Carriage return†		16	010000	
I	i	17	010001	0100 (01)
Δ (delta)	4	18	010010	0100 (10)
?	/	19	010011	0100 (11)
Backspace†		20	010100	
D	d	21	010101	0101 (01)
%	5	22	010110	0101 (10)
]	.	23	010111	0101 (11)
Tab		24	011000	0000 (00)
N	n	25	011001	0110 (01)
$ (dollar)	6	26	011010	0110 (10)
[	,	27	011011	0110 (11)
M	m	29	011101	0111 (01)
∏ (pi)	7	30	011110	0111 (10)
V	v	31	011111	0111 (11)
In: Stop reader Out: ' (prime)		32	100000	
P	p	33	100001	1000 (01)
Σ (sigma)	8	34	100010	1000 (10)
O	o	35	100011	1000 (11)
E	e	37	100101	1001 (01)
(	9	38	100110	1001 (10)
X	x	39	100111	1001 (11)
U	u	41	101001	1010 (01)
F	f	42	101010	1010 (10)
T	t	45	101101	1011 (01)
G	g	46	101110	1011 (10)
H	h	49	110001	1100 (01)
J	j	50	110010	1100 (10)
C	c	53	110101	1101 (01)
K	k	54	110110	1101 (10)
A	a	57	111001	1110 (01)
Q	q	58	111010	1110 (10)
S	s	61	111101	1111 (01)
W	w	62	111110	1111 (10)
Delete†*		63	111111	

† Not used in input

* In this context, "Delete" refers to a paper tape delete code – all columns punched – meaning that the character will be ignored

Alpha Order

Alphanumeric Keys		Output Code		Input Code
Upper Case	**Lower Case**	**Decimal (track)**	**6-bit (track)**	**4-bit (6-bit) binary**
A	a	57	111001	1110 (01)
B	b	5	000101	0001 (01)
C	c	53	110101	1101 (01)
D	d	21	010101	0101 (01)
E	e	37	100101	1001 (01)
F	f	42	101010	1010 (10)
G	g	46	101110	1011 (10)
H	h	49	110001	1100 (01)
I	i	17	010001	0100 (01)
J	j	50	110010	1100 (10)
K	k	54	110110	1101 (10)
M	m	29	011101	0111 (01)
N	n	25	011001	0110 (01)
O	o	35	100011	1000 (11)
P	p	33	100001	1000 (01)
Q	q	58	111010	1110 (10)
R	r	13	001101	0011 (01)
S	s	61	111101	1111 (01)
T	t	45	101101	1011 (01)
U	u	41	101001	1010 (01)
V	v	31	011111	0111 (11)
W	w	62	111110	1111 (10)
X	x	39	100111	1001 (11)
Y	y	9	001001	0010 (01)
Z	z	1	000001	0000 (01)
)	0	2	000010	0000 (10)
L	1	6	000110	0001 (10)
*	2	10	001010	0010 (10)
"	3	14	001110	0011 (10)
Δ (delta)	4	18	010010	0100 (10)
%	5	22	010110	0101 (10)
$ (dollar)	6	26	011010	0110 (10)
∏ (pi)	7	30	011110	0111 (10)
Σ (sigma)	8	34	100010	1000 (10)
(	9	38	100110	1001 (10)
_	-	7	000111	0001 (11)
=	+	11	001011	0010 (11)
:	;	15	001111	0011 (11)
?	/	19	010011	0100 (11)
]	.	23	010111	0101 (11)
[	,	27	011011	0110 (11)
Start reader†		0	000000	
Space		3	000011	0000 (11)
Lower case†		4	000100	
Upper case†		8	001000	
Colour shift†		12	001100	
Carriage return†		16	010000	
Backspace†		20	010100	
Tab		24	011000	0000 (00)
In: Stop reader Out: ' (prime)		32	100000	
Delete†*		63	111111	

† Not used in input

* In this context, "Delete" refers to a paper tape delete code – all columns punched – meaning that the character will be ignored

Appendix 3 –Test Programs

The assembler_tests directory contains programs for testing the assembler as a whole. Output can be checked by reading the listing, or ingesting into the simulator, whichever is considered the more appropriate.

The tests directory contains test modules for pytest testing of the expression parser/evaluator only.

Appendix 4 Demonstration Programs

The demo_programs directory contains the following demonstration programs.

Echo Console Input in 6-bit Mode

Source Code

```
; echo.asm
; Echo characters received from the keyboard in 6-bit mode
; Set mode to 6-bit before running
; Terminate each character with a stop code (')
; Do NOT type more than one character before the stop code
;
.locn 3737              ; any old location
;
start:
    c harmless          ; dump then clear A/ac
    p 0                 ; start proceedings
    i 0                 ; get input - 6 bits in the lowest part of A/ac
    n shift2track       ; left shift bits to track
    a pinstr            ; drop on a p instruction
    h do_print          ; store the instr to be executed
do_print:
    .word 0             ; print
    u start             ; loop
;
shift2track:    .lshn 8 ; value to left shift 8 bits using n mult
pinstr:         p 0     ; p instruction with 0 code
harmless:       .word 0 ; some place to dump A/ac contents
;
.end start
```

Assembly

```
>assembler echo.asm -o echo.hex -l echo.lst
XASM_LGP30 Version 1.0
Listing file: echo.lst
Source file: echo.asm
End of pass 1 - 0 errors
End of pass 2 - 0 errors
Object file: echo.hex
>
```

Execution

```
> simulator echo.hex
SIM_LGP30 version 1.0
Reading hex file: echo.hex
Address: 3737 <-- 866956
(etc.)
(etc.)
Address: 3747 <-- 0
Start address in hex file: 3737
--> tapemode 6
Tape mode: 6
--> run
Expecting keyboard input
None A/ac in c instruction at 3737 - substituting zero
(typed a'b'c'd'<escape>)
abcd
Input terminated by non-Friden character
--> exit
>
```

Hello World Program (print strings)

The hello world program (helloworld.asm) prints strings of characters using a print_string subroutine.

Source Code

```
; helloword.asm
; prints strings using a print_string subroutine
;
; A string comprises a series of 6-bit character codes, up to 5 per word.
; The string is terminated by a stop code (') which is not printed.
; The creator of the string must not make any assumption about case (upper or lower) at the commencement of printing.
; The creator of the string must include case shifts as required to produce the required characters.
; To assemble a string:
; string: .word shl(<code0>,2) + shl(<code1>,8) + shl(<code2>,14) + shl(<code3>,20) + shl(<code4>,26)
; .word shl(<code5>,2) + shl(<code6>,8) + shl(<code7>,14) + etc.
; The first code in each word must reside in the sector field, others must be left shifted in multiples of 6 bits.
;
; Some help from the cross assembler would be good,
; e.g. .string <string_using_escape_codes_where required>,
; but not available at this time.
;
; Calling Convention:
; Put b <address_of_string> in the accumulator (outer argument)
; Plant the return address at the end of the print_string function
; Jump to print_string
;
; Print String Procedure:
; Do until a stop code is found:
;   Print each character in the word
;   Index to the next word
; Return to caller
;
; Notes:
; 1. The current word address, the loop counter, and the incrementer/decrementer are integers at q=29
; 2. The equates hereunder provide the character codes.

        .locn   0                       ; load address
main:
        b       arg1                    ; bring the outer argument to A/ac
        r       print_string_end        ; plant the return address
```

```
        u       print_string            ; jump to subroutine
        b       arg2                    ; returns here - bring the outer argument to A/ac
        r       print_string_end        ; plant the return address
        u       print_string            ; jump to subroutine
        z       0                       ; returns here - end of main program
arg1:
        b       my_1st_string           ; outer argument - brings first word of string
my_1st_string:                          ; Hello World<caret><'>
        .word   shl(ucase,2)+shl(h,8)+shl(lcase,14)+shl(e,20)+shl(l,26)
        .word   shl(l,2)+shl(o,8)+shl(space,14)+shl(ucase,20)+shl(w,26)
        .word   shl(lcase,2)+shl(o,8)+shl(r,14)+shl(l,20)+shl(d,26)
        .word   shl(caret,2)+shl(stop,8)
arg2:
        b       my_2nd_string           ; outer argument - brings first word of string
my_2nd_string:                          ; Good night and good luck<caret><'>
        .word   shl(ucase,2)+shl(g,8)+shl(lcase,14)+shl(o,20)+shl(o,26)
        .word   shl(d,2)+shl(space,8)+shl(n,14)+shl(i,20)+shl(g,26)
        .word   shl(h,2)+shl(t,8)+shl(space,14)+shl(a,20)+shl(n,26)
        .word   shl(d,2)+shl(space,8)+shl(g,14)+shl(o,20)+shl(o,26)
        .word   shl(d,2)+shl(space,8)+shl(l,14)+shl(u,20)+shl(c,26)
        .word   shl(k,2)+shl(caret,8)+shl(stop,14)
                                        ;
print_string:
        h       outer_arg               ; save the outer argument for later execution
                                        ;
inner_loop:
        b       counter_init            ; start_inner_loop - bring the inner loop counter initial value
        h       counter                 ; initialise the inner loop counter
outer_arg:
        .word   0                       ; holds a b instr for execution - brings a word for serialisation to the printer
        h       inner_arg               ; save the inner argument

continue_inner_loop:
        e       code_mask               ; mask in the code bits (in sector field)
        h   code                        ; save the code
        s       stop_code               ; subtract stop code (in sector field)
        t       not_stop_code           ; skip on negative to not_stop_code
        b       stop_code               ; bring stop code (in sector field)
        s       code                    ; subtract code
        t       not_stop_code           ; skip on negative to not_stop_code
        u       print_string_end        ; skip to end_of_procedure (not higher, not lower, then equal)
not_stop_code:                          ; not_stop_code
        b       code                    ; retireve the code
```

```
        n       left_shift_6            ; shift code into the track field (left, 6 bits)
        a       p_instr                 ; add a p instruction (p instruction acquires code)
        h       pinstr_exec             ; save the p instruction for immediate execution
pinstr_exec:
        .word   0                       ; holds a p instr for execution - prints the current code
        b       inner_arg               ; retrieve the inner argument
        m       right_shift_6           ; shift the next character into the sector field (right, six bits)
        h       inner_arg               ; save the inner argument
        b       counter                 ; retrieve the inner loop counter
        s       incdec                  ; subtract indexer
        t       continue_outer_loop     ; skip on negative to continue_outer_loop
        h       counter                 ; save the inner loop counter
        b       inner_arg               ; retrieve the inner_argument
        u       continue_inner_loop     ; jump to inner_loop
                                        ;
continue_outer_loop:
        b       incdec                  ; continue_outer_loop - retrieve the indexer
        a       outer_arg               ; add the outer argument (indexes the address of the word to be serialised to the printer)
        h       outer_arg               ; save the outer argument for later execution
        u       inner_loop              ; repeat the inner loop

print_string_end:
        u       0                       ; end_of_procedure - patched for return to caller
                                        ;
counter:        .word   0               ; the inner loop counter here
inner_arg:          .word       0       ; inner argument here - the word to be serialised to the printer (as shifted)
code:           .word   0               ; the code for printing here (as extracted from inner argument)
counter_init:   .atq    4 29            ; CONST inner loop counter initial value here
code_mask:      .word   0x000000fc      ; CONST 6-bit mask for sector field
stop_code:      .word   shl(stop,2)     ; CONST stop code (in sector field)
left_shift_6:   .lshn   6               ; CONST n multiplier for 6 -bit left shift
p_instr:        p       0               ; CONST p instruction (with track field to be filled)
right_shift_6:  .rshm   6               ; CONST m multiplier for 6-bit right shift
incdec:         .atq    1 29            ; CONST incrementer/decrementer here
                                        ;
                                        ; CONST character codes
        .equ    z       1
        .equ    rbrac   2
        .equ    n0      2
        .equ    space   3
        .equ    lcase   4
        .equ    b       5
        .equ    l       6
```

```
.equ    n1      6
.equ    uscor   7
.equ    minus   7
.equ    ucase   8
.equ    y       9
.equ    mul     10
.equ    n2      10
.equ    eql     11
.equ    plus    11
.equ    colour  12
.equ    r       13
.equ    quot    14
.equ    n3      14
.equ    coln    15
.equ    scoln   15
.equ    caret   16
.equ    i       17
.equ    delta   18
.equ    n4      18
.equ    ques    19
.equ    div     19
.equ    bs      20
.equ    d       21
.equ    pct     22
.equ    n5      22
.equ    rsb     23
.equ    fstop   23
.equ    tab     24
.equ    n       25
.equ    dol     26
.equ    n6      26
.equ    lsb     27
.equ    comma   27
.equ    m       29
.equ    pi      30
.equ    n7      30
.equ    v       31
.equ    stop    32
.equ    p       33
.equ    sigma   34
.equ    n8      34
.equ    o       35
.equ    e       37
```

```
.equ    lbrac   38
.equ    n9      38
.equ    x       39
.equ    u       41
.equ    f       42
.equ    t       45
.equ    g       46
.equ    h       49
.equ    j       50
.equ    c       53
.equ    k       54
.equ    a       57
.equ    q       58
.equ    s       61
.equ    w       62
                                ;
.end    main
```

Assembly

```
> assembler helloworld.asm -o helloworld.hex -l helloworld.lst
XASM_LGP30 Version 1.0
Listing file: helloworld.lst
Source file: helloworld.asm
End of pass 1 - 0 errors
End of pass 2 - 0 errors
Object file: helloworld.hex
>
```

Execution

```
> simulator helloworld.hex
SIM_LGP30 version 1.0
Reading hex file: helloworld.hex
Address: 0 <-- 65564
(etc.)
(etc.)
Address: 60 <-- 4
Start address in hex file: 0
--> run
Expecting keyboard input
Hello World
Good night and good luck
--> exit
>
```

Evaluate a Polynomial Expression

Source Code

```
; polynomial.asm
; Evaluate polynomial
; LGP-30 Programming Manual, April 1957, Pages 12-20
; Page 19, 'Final Program', excluding call to print routine
;
; Calcuate y = a0*x^4 + a1*x^3 + a2*x^2 + a3*x + a4
; Refactored as y = (((a0*x + a1)*x + a2)*x + a3)*x + a4
;
.locn 1000
start:
    b add_instr ; initial add instr
    c do_add    ; place inline and zero A/ac
    h result    ; clear intermediate value
;
loop:
    b result    ; get intermediate value
    m x         ; * x (initially 0)
do_add:
    .word 0     ; indexed add instr for execution
    h result    ; update intermediate value
;
    b do_add    ; get add instr
    a CONST_1   ; increment address
    h do_add    ; update add instr
    s end_instr ; check for end of func
    t loop      ; go back for more?
    z 0         ; end of prog - temp contains result
;
result:
    .word 0     ; result accumulated here - q value of a0*x^4
CONST_1:
    .atq 1 29   ; incrementer
add_instr:
    a a0        ; add instr with addr of first constant
end_instr:
    a end_add   ; add instr with addr past last constant
;
x:  .word 0     ; put x here using atqdeposit
a0: .word 0     ; put a0 here using atqdeposit
a1: .word 0     ; put a1 here - q value of a0*x using atqdeposit
a2: .word 0     ; put a2 here - a value of a0*x^2 using atqdeposit
a3: .word 0     ; put a3 here - q value of a0*x^3 using atqdeposit
a4: .word 0     ; put a4 here - q value of a0*x^4 using atqdeposit
end_add: .word 0    ; past last constant
;
.end start

; some test values
; x   1.2 at q = 3      -1.2 at q = 4
; a0  3.4 at q = 3      -4.3 at q = 5
; a1  5.6 at q = 6      -5.6 at q = 9
; a2  7.8 at q = 9      -8.7 at q = 13
; a3  9.0 at q = 12     -9.0 at q = 17
; a4  1.2 at q = 15     -2.1 at q = 21
; y   39.95904          -3.06768
```

Assembly

```
> assembler polynomial.asm –o polynomial.hex –l polynomial.lst
XASM_LGP30 Version 1.0
Listing file: polynomial.lst
Source file: polynomial.asm
End of pass 1 – 0 errors
End of pass 2 – 0 errors
Object file: polynomial.hex
>
```

Execution

See listing file for addresses and appropriate q values of x, coefficients and result.

```
> simulator polynomial.hex
SIM_LGP30 version 1.0
Reading hex file: polynomial.hex
Address: 1000 <-- 69596
(etc.)
(etc.)
Address: 1023 <-- 0
Start address in hex file: 1000
--> atqdeposit 1017 1.2 3
1017: 1.1999999955296516 3
--> atqdeposit 1018 3.4 3
1018: 3.399999998509884 3
--> atqdeposit 1019 5.6 6
1019: 5.599999964237213 6
--> atqdeposit 1020 7.8 9
1020: 7.799999713897705 9
--> atqdeposit 1021 9.0 12
1021: 9.0 12
--> atqdeposit 1022 1.2 15
1022: 1.199981689453125 15
--> run
Expecting keyboard input
--> atqexamine 1013 15
1013: 39.959014892578125 15
--> exit
>
```